

## BUỔI 11: Kiểu tệp tin (file) – Tệp tin nhị phân

### I. Lý thuyết

#### 1. Định nghĩa

Tệp tin nhị phân xem dữ liệu như là một dãy byte liên tục, việc xử lý với tệp tin dựa trên việc đọc ghi dãy byte.

Một số vấn đề cần lưu ý với file nhị phân:

- **Biến tệp tin:** là một biến thuộc kiểu dữ liệu tệp tin dùng để đại diện cho một tệp tin. Dữ liệu chứa trong một tệp tin được truy xuất qua các thao tác với thông số là biến tệp tin đại diện cho tệp tin đó.
- **Con trỏ tệp tin:** Khi một tệp tin được mở ra để làm việc, tại mỗi thời điểm, sẽ có một vị trí của tệp tin mà tại đó việc đọc/ghi thông tin sẽ xảy ra. Người ta hình dung có một con trỏ đang chỉ đến vị trí đó và đặt tên nó là con trỏ tệp tin.
- Sau khi đọc/ghi xong dữ liệu, con trỏ sẽ chuyển dịch thêm một phần tử về phía cuối tệp tin. Sau phần tử dữ liệu cuối cùng của tệp tin là dấu kết thúc tệp tin EOF (End Of File). Tệp tin nhị phân thường được sử dụng khi muốn lưu trữ các khối dữ liệu, trong đó mỗi khối bao gồm các byte liên tục. Mỗi khối biểu diễn một cấu trúc phức tạp hoặc một mảng. Ví dụ, dữ liệu gồm một cấu trúc nhiều thành phần, thay vì đọc từng thành phần đọc lập người ta thường đọc/ghi toàn bộ khối dữ liệu vào tệp tin.

#### 2. Làm việc với file nhị phân

+ **Mở file:** Giống với khai báo file văn bản, chỉ khác chế độ mở.

Để mở một file làm việc ta sử dụng khai báo sau:

```
FILE *fopen( const char * ten_file, const char * che_do );
```

Hoặc:

```
FILE *<con_tro_file>  
Con_tro_file = fopen( const char * ten_file, const char * che_do );
```

Trong đó:

- **Ten\_file:** là một chuỗi ký tự đại diện cho tên của file đã có hoặc tạo mới. Nếu không chỉ ra đường dẫn chứa file thì chương trình sẽ ngầm hiểu file này ở trong thư mục chứa mã nguồn của project.
- **Che\_do:** là các chế độ xử lý file, bao gồm :

| Chế độ | Giải thích                                                                                                                                                                     |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rb     | Mở file nhị phân đã tồn tại với mục đích đọc                                                                                                                                   |
| wb     | Mở file nhị phân đã tồn tại với mục đích ghi. Nếu các file này chưa tồn tại thì file mới sẽ được tạo ra. Ở đây, chương trình bắt đầu ghi nội dung từ phần mở đầu của file.     |
| ab     | Mở file nhị phân cho việc ghi ở chế độ ghi tiếp theo vào cuối, nếu file chưa tồn tại thì file mới được tạo ra. Ở đây, chương trình ghi nội dung với phần cuối file đã tồn tại. |
| rb+    | Mở file nhị phân đã tồn tại với mục đích đọc và ghi                                                                                                                            |
| wb+    | Mở file nhị phân đã tồn tại với mục đích đọc và ghi. Nếu các file này đã có thì nó sẽ xóa sạch các dữ liệu và nếu file chưa tồn tại thì file mới sẽ được tạo ra.               |
| ab+    | Mở file nhị phân đã tồn tại với mục đích đọc và ghi. Nó tạo file mới nếu không tồn tại. Việc đọc file sẽ bắt đầu đọc từ đầu nhưng ghi file sẽ chỉ ghi vào cuối file.           |

### **+Ghi File:**

Để ghi nội dung vào file nhị phân, ta sử dụng hàm fwrite() với cú pháp như sau:

```
size_t fwrite(const void *ptr, size_t size, size_t n, FILE *f)
```

Trong đó:

- ptr: con trỏ chỉ đến vùng nhớ chứa thông tin cần ghi lên tập tin.
- n: số phần tử sẽ ghi lên tập tin.
- size: kích thước của mỗi phần tử.
- f: con trỏ tập tin đã được mở.

Giá trị trả về của hàm này là số phần tử được ghi lên tập tin. Giá trị này bằng n trừ khi xuất hiện lỗi.

### **+ Đọc File:**

Để đọc nội dung từ file nhị phân, ta sử dụng hàm fread() với cú pháp như sau:

```
size_t fread(const void *ptr, size_t size, size_t n, FILE *f)
```

Trong đó:

- ptr: con trỏ chỉ đến vùng nhớ sẽ nhận dữ liệu từ tập tin.
- n: số phần tử được đọc từ tập tin.
- size: kích thước của mỗi phần tử.
- f: con trỏ tập tin đã được mở.

Giá trị trả về của hàm này là số phần tử đã đọc được từ tập tin. Giá trị này bằng n hay nhỏ hơn n nếu đã chạm đến cuối tập tin hoặc có lỗi xuất hiện.

### **+ Di chuyển con trỏ tập tin về đầu tập tin:**

Để di chuyển con trỏ về đầu tệp tin, ta sử dụng hàm rewind()

```
void rewind(FILE *fp)
```

### **+ Di chuyển con trỏ tệp tin về vị trí bất kỳ:**

Để di chuyển con trỏ đến vị trí bất kỳ, ta sử dụng hàm fseek()

```
int fseek(FILE *stream, long offset, int whence);
```

Trong đó:

- stream: con trỏ tập tin đang thao tác.
- offset: số byte cần dịch chuyển con trỏ tập tin.
- whence: vị trí bắt đầu để tính khoảng cách dịch chuyển cho offset:

|   |          |                                 |
|---|----------|---------------------------------|
| 0 | SEEK_SET | Vị trí đầu tệp                  |
| 1 | SEEK_CUR | Vị trí hiện tại của con trỏ tệp |
| 2 | SEEK_END | Vị trí cuối tệp tin             |

Kết quả trả về của hàm là 0 nếu việc di chuyển thành công. Nếu không thành công, 1 giá trị khác 0 (đó là 1 mã lỗi) được trả về.

**Ví dụ 1:** Viết chương trình ghi lên tệp tin input1.txt 3 giá trị (nguyên, thực, chuỗi). Sau đó, đọc từ tệp tin các giá trị này và in ra màn hình.

```
int main()
{
    FILE *fp;
    double x=23.31,x1;
    int y=13,y1;
    char s[10]="Xin chao",s1[20];
    fp=fopen("File1","wb+");
    if (fp==NULL)
        printf("Cannot open file");
    else
    {
        fwrite(&x,sizeof(double),1,fp);
        fwrite(&y,sizeof(int),1,fp);
        fwrite(&s,sizeof(char),10,fp);
        rewind(fp); //Đưa con trỏ về đầu file
        fread(&x1,sizeof(double),1,fp);
        fread(&y1,sizeof(int),1,fp);
        fread(&s1,sizeof(char),20,fp);
        printf("x1=%5.2f, y1=%d, s1=%s",x1,y1,s1);
        fclose(fp);
    }
    return 0;
}
```

**Ví dụ 2:** Một sinh viên bao gồm ba thông tin: mã sinh viên, họ tên và điểm. Viết chương trình cho phép lựa chọn các chức năng:

1. Nhập danh sách sinh viên từ bàn phím rồi ghi lên tệp tin SinhVien.dat
2. Đọc dữ liệu từ tệp tin SinhVien.dat rồi hiển thị danh sách lên màn hình
3. Tìm kiếm họ tên của một sinh viên nào đó dựa vào mã sinh viên nhập từ bàn phím.
4. Thoát

## ***Hướng dẫn***

```
typedef struct
{
    int MaSV;
    char hoten[30];
    float diem;
}SV;
void Write(FILE *fp)
{
    SV x;
```

```

int i, n;
fp=fopen("input.txt","ab");
printf("Nhap vao so luong sinh vien:");
scanf("%d",&n);
for (i=1;i<=n;i++)
{
    printf("Nhap sinh vien thu %d",i);
    printf("\nMaSV:");scanf("%d",&x.MaSV);
    printf("Ho ten:");fflush(stdin);gets(x.hoten);
    printf("Diem:");scanf("%f",&x.diem);
    fwrite(&x,sizeof(SV),1,fp);
}
fclose(fp);
}
void Read(FILE *fp)
{
    SV x;
    fp=fopen("input.txt","rb");
    if (fp==NULL)
        printf("Cannot open file");
    else
    {
        printf("\nMaSV      |Ho Ten          |Diem");
        fread(&x,sizeof(SV),1,fp);
        while (!feof(fp))
        {
            printf("\n%d      |%s              |%5.2f",x.MaSV,x.hoten,x.diem);
            fread(&x,sizeof(SV),1,fp);
        }
        fclose(fp);
    }
}

void Search(FILE *fp)
{
    int x,check=0;
    SV sv;
    printf("Nhap ma SV can tim:");scanf("%d",&x);
    fp=fopen("input.txt","rb");
    if (fp==NULL)
    {
        printf("Cannot open file");
        exit(1);
    }
    fread(&sv,sizeof(SV),1,fp);
    while (!feof(fp))
    {
        if (sv.MaSV==x)
        {
            printf("\nThong tin ve sinh vien can tim la:\n");
            printf("%d-%s-%5.2f",sv.MaSV,sv.hoten,sv.diem);
            check=1;
            break;
        }
        fread(&sv,sizeof(SV),1,fp);
    }
    fclose(fp);
    if(check==0) printf("\nKhong tim thay sinh vien co ma vua nhap!");
}
void Menu()

```

```

{
    system("cls");
    printf("CHUONG TRINH FILE QUAN LY SINH VIEN\n");
    printf("1. Ghi sinh vien vao File\n");
    printf("2. Doc danh sach Sinh vien tu file\n");
    printf("3. Tim kiem sinh vien theo ma sinh vien\n");
    printf("4. Thoat\n");
}

int main()
{
    char c,x[20];
    FILE *fp;
    do
    {
        Menu();
        printf("Moi ban nhap lua chon:");
        scanf("%c",&c);
        switch (c)
        {
            case '1': Write(fp);break;
            case '2': Read(fp);break;
            case '3': Search(fp);break;
            case '4': return 0;
            default: printf("\nBan da nhap sai lua chon, moi nhap lai!\n");
        }
        fflush(stdin);
        getch();
    }while (1);
    return 0;
}

```

## II. Bài tập

**Câu 35:** Viết chương trình sử dụng kiểu nhập xuất nhị phân với tệp có tên là SO\_LIEU.C. Số liệu bán hàng có cấu trúc gồm các thành phần: Ten\_hang, Don\_gia, So\_luong, Thanh\_tien (= So\_luong \* Don\_gia). Sau đó hiện nội dung tệp lên màn hình theo dạng:

### SO LIEU BAN HANG

| STT       | Ten Hang | Don gia | So luong | Thanh tien |
|-----------|----------|---------|----------|------------|
| 1         | Sach     | 5       | 100      | 500        |
| 2         | But      | 2       | 300      | 600        |
| Tong tien |          |         |          | 1100       |

**Câu 36:** Viết chương trình sử dụng kiểu nhập xuất nhị phân với tệp có tên là DSACH.C để ghi danh sách các sinh viên có cấu trúc gồm các thành phần: Ho\_ten, Tuoi, Diem\_TB.

- Hiện nội dung tệp lên màn hình theo dạng:

### DANH SACH SINH VIEN

| STT | Ho ten | Tuoi | Diem TB |
|-----|--------|------|---------|
| ... | ...    |      |         |

- Bổ sung thêm một sinh viên (nhập vào từ bàn phím) vào cuối tệp, rồi in lại danh sách theo dạng trên.